

实验6 循环程序设计

一.实验目的

理解循环程序结构的特点，掌握循环结构程序的编写。

二.实验内容

编写循环结构的斐波那契数列程序，输出斐波那契数。

编写求最大N值的自然数求和程序（循环结构），具体要求是：进行自然数相加（ $1 + 2 + 3 + \dots + N$ ），无符号整数的累加和用一个32位寄存器存储，求出（显示）有效累加和的最大值及对应的最大N值。

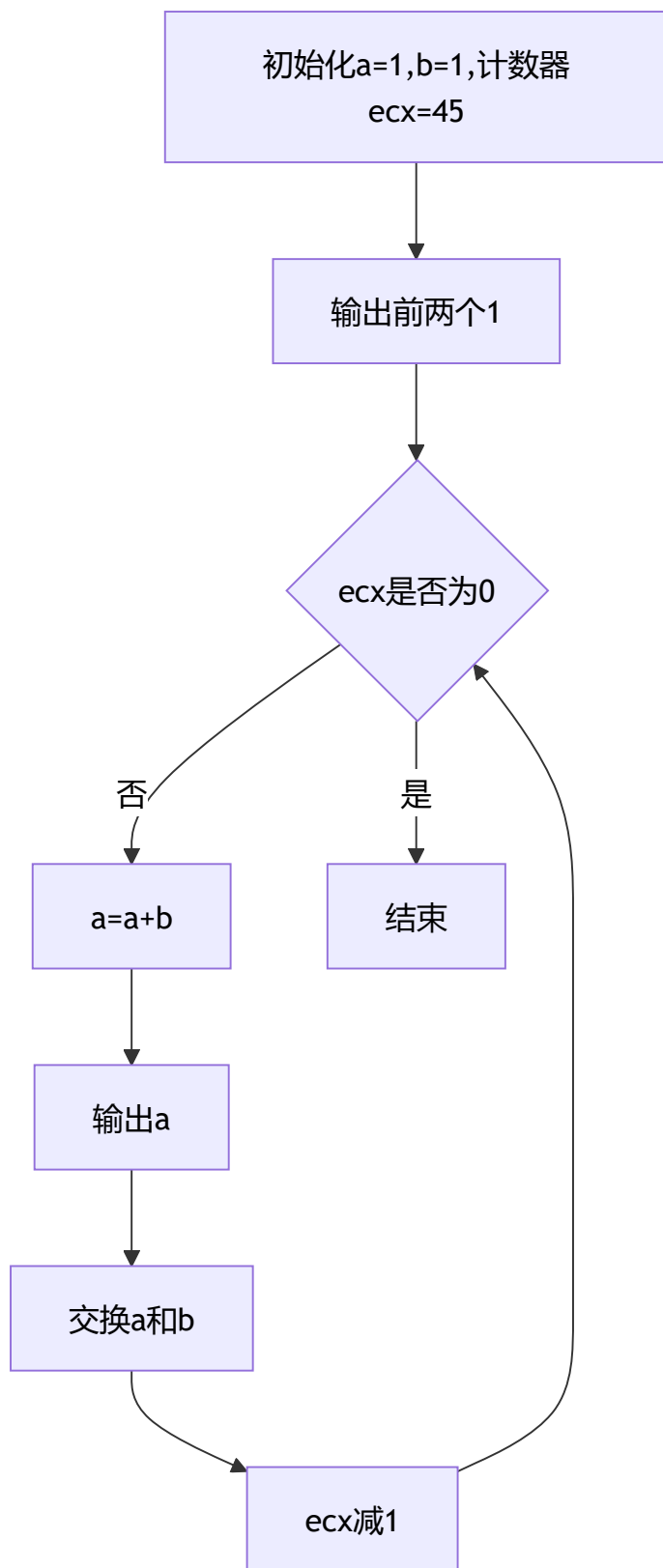
编写多重循环的冒泡法排序程序，并运行正确。

编写显示ASCII表程序，并运行正确。

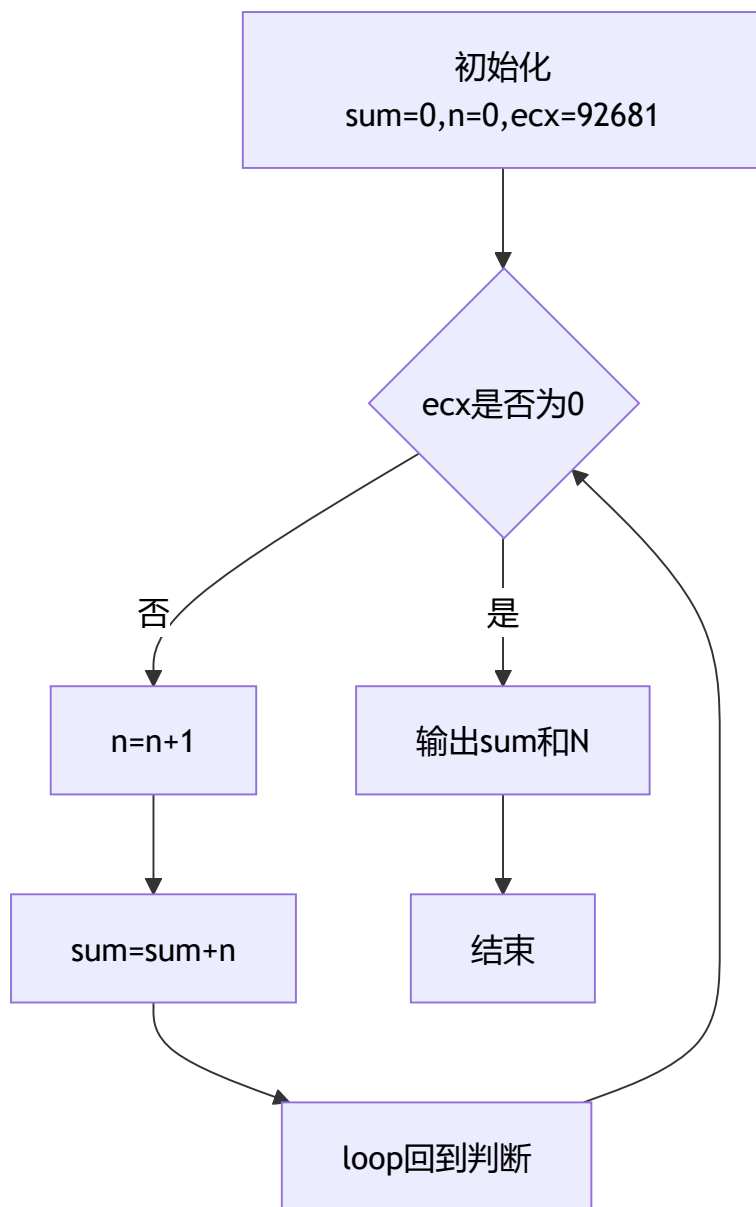
三、实验分析

画出自己编写实现的实验内容（1）、（2）、（3）和（4）的程序流程图，给出核心代码、注释以及运行结果截图。

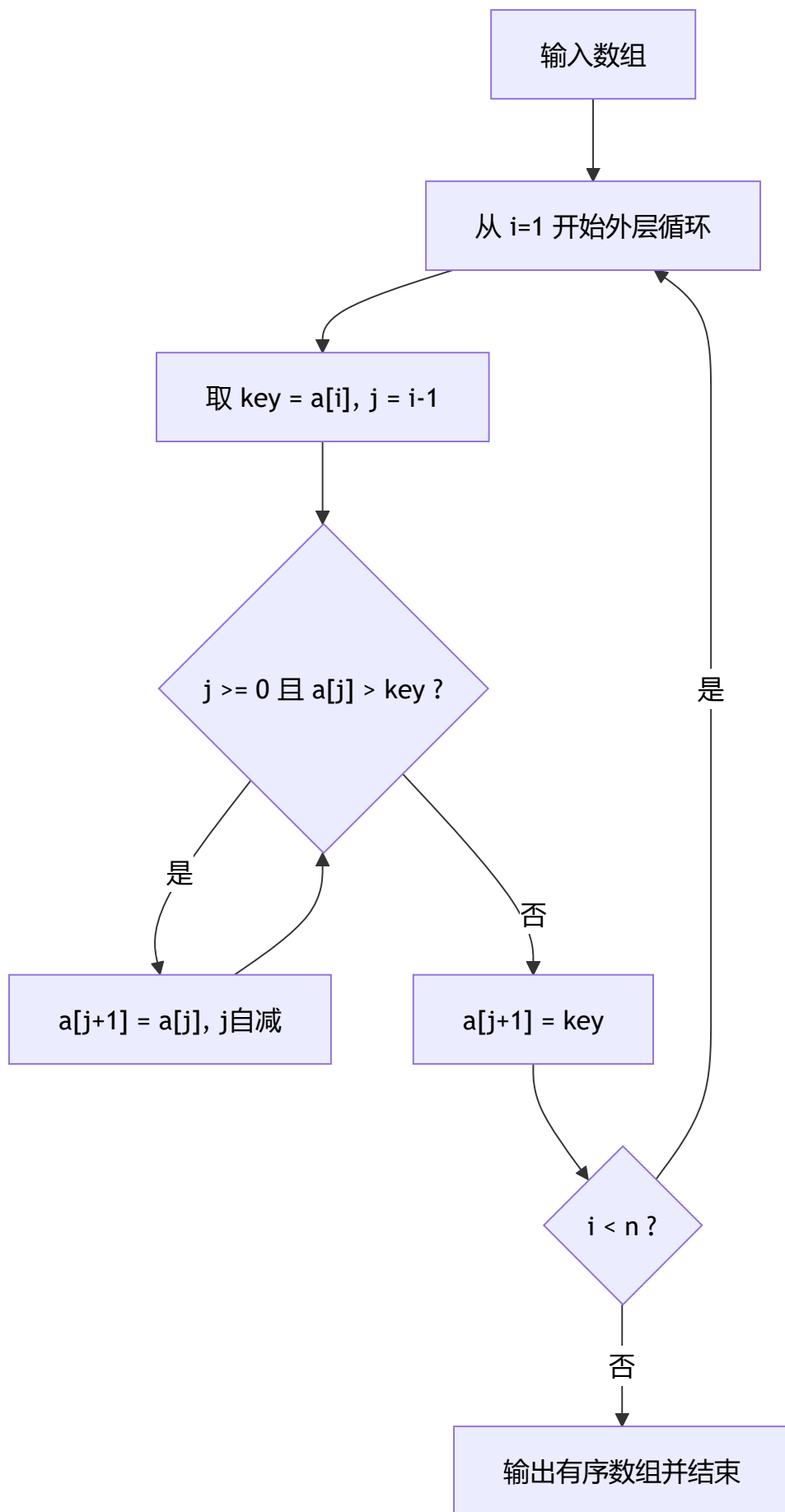
1. 实验内容（1）斐波那契数列流程图（exp0602.s）



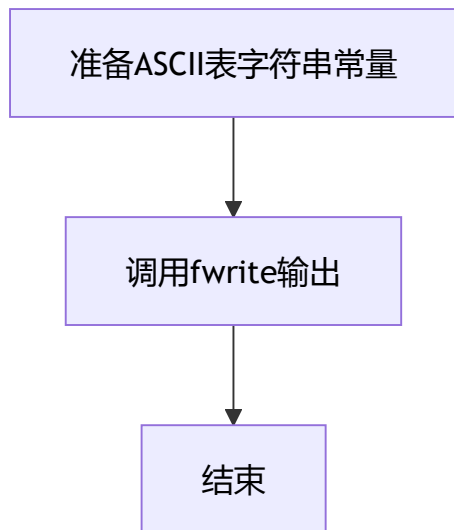
2. 实验内容 (2) 自然数求和最大N流程图 (exp0601.s)



3. 实验内容 (3) 排序程序流程图 (exp0603)



4. 实验内容 (4) ASCII表显示流程图 (exp0604)



5. 核心代码及注释

(1) 斐波那契循环核心 (exp0602.s)

```
.section .data

.section .text
.globl main
.type main, @function
```

```
main:
    subq $40, %rsp
    movl $1, %eax
    call dispuid
    call dispctrlf
    movl $1, %eax
    call dispuid
    call dispctrlf
    movl $1, %eax
    movl $1, %ebx
    movl $45, %ecx
```

```
Fib:
    addl %ebx, %eax
    push %rcx
    push %rbx
    push %rax
    call dispuid
    call dispctrlf
    pop %rax
    pop %rbx
    pop %rcx
    xchgl %eax, %ebx
    loop Fib
```

```
Done:
    addq $40, %rsp
    xorl %eax, %eax
    ret
```

(2) 自然数求和核心 (exp0601.s)

```
.section .data

.section .text
.globl main
.type main, @function

main:
    xorl %eax, %eax
    xorl %ebx, %ebx
    movl $92681, %ecx

Sum:
    incl %ebx
    addl %ebx, %eax
    loop Sum

Done:
    call dispuid
    call dispCrLf
    movl $92681, %eax
    call dispuid
    call dispCrLf

    xorl %eax, %eax
    ret
```

(3) 排序核心 (exp0603 , 由C编译得到汇编)

```
for (i = 1; i < n; i++) {
    key = a[i];
    j = i - 1;
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j];
        j--;
    }
    a[j + 1] = key;
}
```

(4) ASCII表显示核心 (exp0604)

```
.file      "exp0604.c"
.intel_syntax noprefix
.section   .rodata
.align    8

.LC0:
.ascii    "10 | 0 1 2 3 4 5 6 7 8 9 A B C D "
.string    "E F\n-- + -----\n20 |  ! \" # $ % & ' ( ) * +
.text
.globl    main
.type     main, @function

main:
.LFB0:
.cfi_startproc
push      rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
mov       rbp, rsp
.cfi_def_cfa_register 6
mov       rax, QWORD PTR stdout[rip]
mov       rcx, rax
mov       edx, 289
mov       esi, 1
mov       edi, OFFSET FLAT:.LC0
call      fwrite
mov       eax, 0
pop       rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc

.LFE0:
.size     main, .-main
.ident    "GCC: (Ubuntu 5.4.0-6ubuntu1~16.04.12) 5.4.0 20160609"
.section  .note.GNU-stack,"",@progbits
```

6. 程序运行结果

(1) exp0601 输出

4294930221
92681

说明：在32位无符号累加范围内，最大有效累加和为 4294930221，对应最大 N=92681。

(2) exp0602 输出 (部分)

```
1
1
2
3
5
8
...
1134903170
1836311903
2971215073
```

说明：斐波那契序列按循环正确输出。

(3) exp0603 输出

```
587
-632
777
234
-34
-632
-34
234
587
777
```

说明：前5行为原数组，后5行为升序结果，排序正确。

(4) exp0604 输出 (节选)

```
10 | 0 1 2 3 4 5 6 7 8 9 A B C D E F
-- + -----
20 | ! " # $ % & ' ( ) * + , - . /
...
70 | p q r s t u v w x y z { | } ~
```

说明：ASCII可显示字符区间排版正确。

四、实验总结

(1) 总结实验中遇到的问题及解决方法

- 循环边界最容易出错，尤其是 `loop` 指令会自动修改 `ecx`。解决方法是先确定循环次数，再把“初始化、循环体、终止条件”分块写清。
- 斐波那契程序中输出函数调用会破坏部分寄存器，解决方法是调用前后 `push/pop` 保护现场。
- 在排序程序阅读汇编时，地址计算（如 `base + index*4`）容易混淆，通过先对照C版本再看汇编，可快速定位每条指令含义。
- ASCII表程序虽然逻辑简单，但让我理解了“循环不一定必须显式写在当前源码中”，也可以提前构造数据后一次性输出。

对比高级语言，总结汇编语言循环程序结构的特点和编程体会。

- 高级语言循环强调语义（`for/while`），汇编循环强调控制细节（计数寄存器、条件码、跳转目标）。
- 汇编循环执行路径清晰、可精确控制性能，但可读性差，维护难度高于高级语言。
- 在汇编里，循环变量、临时变量、函数调用现场都要显式管理，编程时必须养成“先设计流程图再编码”的习惯。
- 对寄存器生命周期和数据宽度（8/16/32/64位）的理解，是写对循环程序的关键。